

Java OOPs Basics

Class → Object → Method → Anonymous Object

1. What is a Class?

A **class** is a blueprint/template for creating objects.

For example, imagine a blueprint for a car:

```
Car Blueprint
-----
color
brand
price

start()
stop()
```

In Java:

```
class Car {

    String color;
    String brand;
    int price;

    void start() {
        System.out.println("Car is starting");
    }

    void stop() {
        System.out.println("Car is stopping");
    }
}
```

Here, `class Car` is the class. It describes what a Car object should have and what it can do.

Class contains mainly:

```
Class
|-- Variables / Data
\-- Methods / Behavior
```

2. What is an Object?

An **object** is a real instance of a class.

Class = blueprint Object = actual thing created from blueprint

We create an object using `new`.

```
Car c1 = new Car();
```

Let's break this down:

```
Car c1 = new Car();
  |      |
  |      \-- Object created
  |
  \-- Reference variable
```

Car — the class/type. c1 — the reference variable. new Car() — creates a new Car object in memory.

3. Complete Example

```
class Car {

    String color;
    String brand;

    void start() {
        System.out.println("Car is starting");
    }

    void display() {
        System.out.println("Color: " + color);
        System.out.println("Brand: " + brand);
    }
}

public class Main {

    public static void main(String[] args) {

        Car c1 = new Car();

        c1.color = "Red";
        c1.brand = "BMW";

        c1.start();
        c1.display();
    }
}
```

Output:

```
Car is starting
Color: Red
Brand: BMW
```

4. What Happens in Memory?

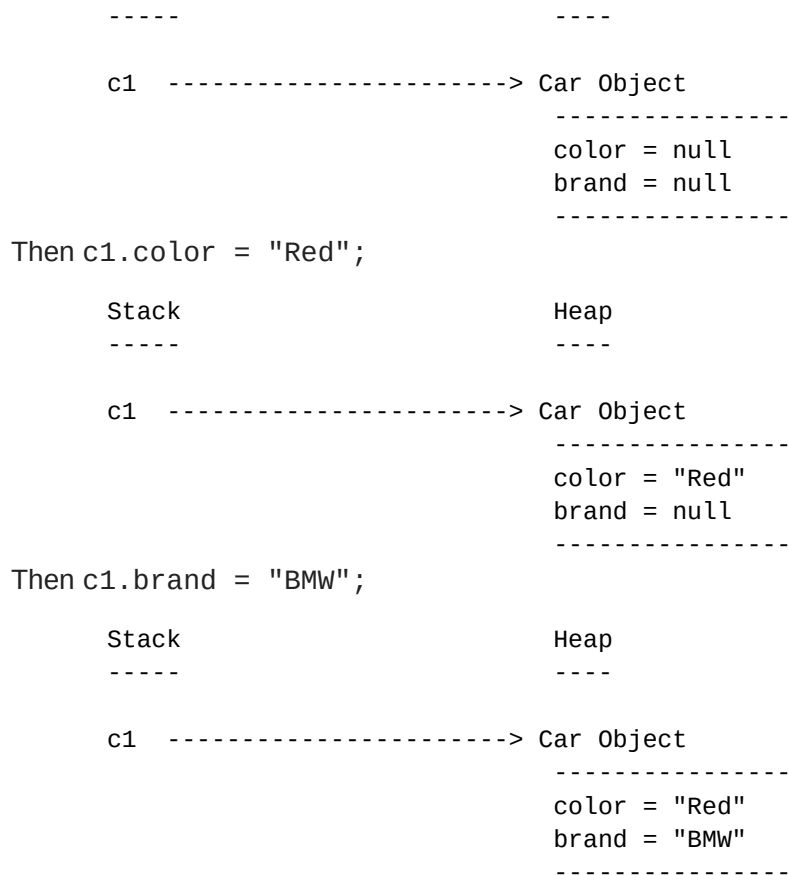
When this executes:

```
Car c1 = new Car();
```

Java creates an object. Conceptually:

Stack

Heap



5. What is a Method?

A **method** is a block of code that performs a task/behavior. Example:

```

void start() {
    System.out.println("Car is starting");
}
void          -> return type
start         -> method name
()            -> parameters
{ }           -> method body

```

We call the method using the object:

```

c1.start();
Object
|
v
c1
|
v
start()

```

6. Why Do We Need Objects to Call Methods?

```

class Student {

```

```
        void study() {  
            System.out.println("Student is studying");  
        }  
    }
```

To call study():

```
Student s1 = new Student();
```

```
s1.study();
```

Why? Because study() belongs to the Student object.

```
Student class  
  |  
  v  
new Student()  
  |  
  v  
object  
  |  
  v  
s1.study()
```

7. Multiple Objects

One class can create many objects.

```
class Student {  
    String name;  
  
    void display() {  
        System.out.println(name);  
    }  
}  
Student s1 = new Student();  
Student s2 = new Student();  
Student s3 = new Student();  
Student class  
|  
|-----> s1 object  
|  
|-----> s2 object  
|  
|-----> s3 object
```

Each object has its own data.

```
s1.name = "Ravi";  
s2.name = "Kiran";  
s3.name = "John";  
  
s1.display();  
s2.display();  
s3.display();
```

Output:

```
Ravi  
Kiran  
John
```

8. What is an Anonymous Object?

Now the important part. Normally we create an object and give it a reference:

```
Car c1 = new Car();
```

Here, Object → c1. But we can create an object without storing it in a reference variable. That is called an **anonymous object**.

```
new Car();
```

There is no reference variable.

```
new Car()  
|  
v  
Object  
No: Car c1
```

9. Anonymous Object Example

```
class Car {  
    void start() {  
        System.out.println("Car is starting");  
    }  
}
```

Normal object:

```
Car c1 = new Car();
```

```
c1.start();
```

Anonymous object:

```
new Car().start();
```

Both can call start().

Normal:

```
Car c1 = new Car();  
c1.start();
```

Anonymous:

```
new Car().start();
```

10. Why Is It Called Anonymous?

Because the object has no reference name.

Normal: `Car c1 = new Car();` — the object can be accessed through `c1`.

Anonymous: `new Car().start();` — there is no name like `c1`.

Anonymous object = object without a reference variable.

11. Important Difference

Normal Object

```
Car c1 = new Car();
```

```
c1.start();  
c1.stop();  
c1.display();
```

You can reuse the same object:

```
    c1  
    |  
    v  
  Car Object  
  /  |  \  
 /   |   \  
start stop display
```

Anonymous Object

```
new Car().start();
```

You can immediately use the object, but you don't have a reference variable to reuse that same object later. For example, `new Car().start();` then `new Car().stop();` creates *another* Car object.

12. When Is Anonymous Object Useful?

Anonymous objects are useful when you need an object only once.

```
class Calculator {  
  
    int add(int a, int b) {  
        return a + b;  
    }  
}
```

Normal:

```
Calculator c = new Calculator();  
  
System.out.println(c.add(10, 20));
```

Anonymous:

```
System.out.println(new Calculator().add(10, 20));
```

Output: 30 — since we only need the calculator once, an anonymous object is convenient.

13. One Complete Example

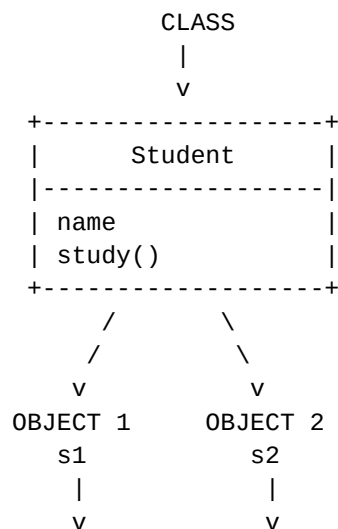
```
class Student {  
  
    String name;  
  
    void study() {  
        System.out.println(name + " is studying");  
    }  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        // Normal object  
        Student s1 = new Student();  
  
        s1.name = "Ravi";  
  
        s1.study();  
  
        // Anonymous object  
        new Student().study();  
    }  
}
```

Output:

```
Ravi is studying  
null is studying
```

Why null? Because `new Student()` creates a new Student object, but we never assigned name. So it has its default value: `null`.

14. Remember This Diagram



name=Ravi name=Kiran

And:

```
Student s1 = new Student();
```

Student -> type/class
s1 -> reference variable
new -> creates object
Student() -> constructor call

Anonymous:

```
new Student().study();
```

```
new Student()  
  |  
  v  
creates object  
  |  
  v  
  .study()  
  |  
  v  
method executes
```

Very Important Summary

Concept	Meaning	Example
Class	Blueprint/template	class Student { }
Object	Instance of class	new Student()
Reference	Variable that refers to object	Student s1
Method	Behavior/action	s1.study()
Anonymous Object	Object without reference variable	new Student().study()

The basic flow to remember:

```
CLASS  
  |  
  v  
new  
  |  
  v  
OBJECT  
  |  
  v  
REFERENCE  
  |  
  v  
METHOD CALL
```

Example:

```
Student s1 = new Student();  
s1.study();
```

```
// anonymous object  
new Student().study();
```

*Next OOP concept to learn: **Constructor** — it connects directly to new Student () and explains exactly what happens when an object is created.*